

ECE 375L Introduction to Computer Architecture



Final Project

8-Bit Microprocessor with Instruction Decoder and ALU

Joey Mei

April 2026

1 Abstract

This project presents the design and implementation of an 8-bit microprocessor featuring an arithmetic logic unit (ALU) and an instruction decoder. The microprocessor is capable of executing fundamental operations such as addition, subtraction, logical operations, and simple data transfers. While 8-bit microprocessors are often viewed as legacy technology, they remain highly relevant in modern applications. Their simplicity, low cost, and energy efficiency make them ideal for embedded systems in home appliances, automotive control units, industrial automation, low-power IoT devices, and educational kits. By implementing a functional 8-bit processor in a laboratory environment, this project demonstrates both the fundamental principles of microprocessor architecture and the enduring utility of 8-bit systems in real-world scenarios.

2 Design Implementation

2.1 Architecture Overview

This project implements a simplified 8-bit microprocessor designed to demonstrate core computer architecture concepts, including instruction decoding, register-based data storage, and arithmetic/logic operations. The system operates without memory; instead, instructions and data are provided externally, allowing the design to focus on datapath execution and control logic.

The architecture consists of several key components: a 4×16 instruction decoder, three 8-bit registers, control logic, and output display modules. Instructions are supplied via a 4-bit opcode input and are executed sequentially using a debounced enable signal derived from a push button.

The 4×16 decoder translates the 4-bit opcode into a one-hot 16-bit control signal. Each output line of the decoder corresponds to a specific instruction, such as data movement, arithmetic operations, logical operations, and shift operations. This one-hot encoding simplifies control logic by ensuring that only one operation is active at a time.

The datapath is centered around three registers. RegA and RegB serve as primary operand registers, while RegC stores the results of arithmetic and logical operations. The Arithmetic Logic Unit (ALU) functionality is implemented within the main sequential logic block, where operations are selected based on decoder outputs. Arithmetic operations produce both an 8-bit result and a carry-out flag, while logical operations directly update the designated register.

At a high level, the system operates in a single-cycle manner. When the enable button is pressed, the input opcode is decoded, generating a control signal that triggers the corresponding operation. Data is transferred between registers or processed by the ALU within one clock cycle. Because no memory is implemented, there is no instruction fetch stage; all instructions are executed directly from external input.

To improve usability, a debounce module is included to ensure clean, single-cycle enable pulses from the mechanical button input. The processor's internal state can be observed through LEDs and a multiplexed 7-segment display, which allows the user to select and view the contents of different registers.

This architecture is intentionally minimal, emphasizing clarity of data flow and control. By eliminating memory and pipelining, the design highlights how instructions are decoded and executed within a basic microprocessor system.

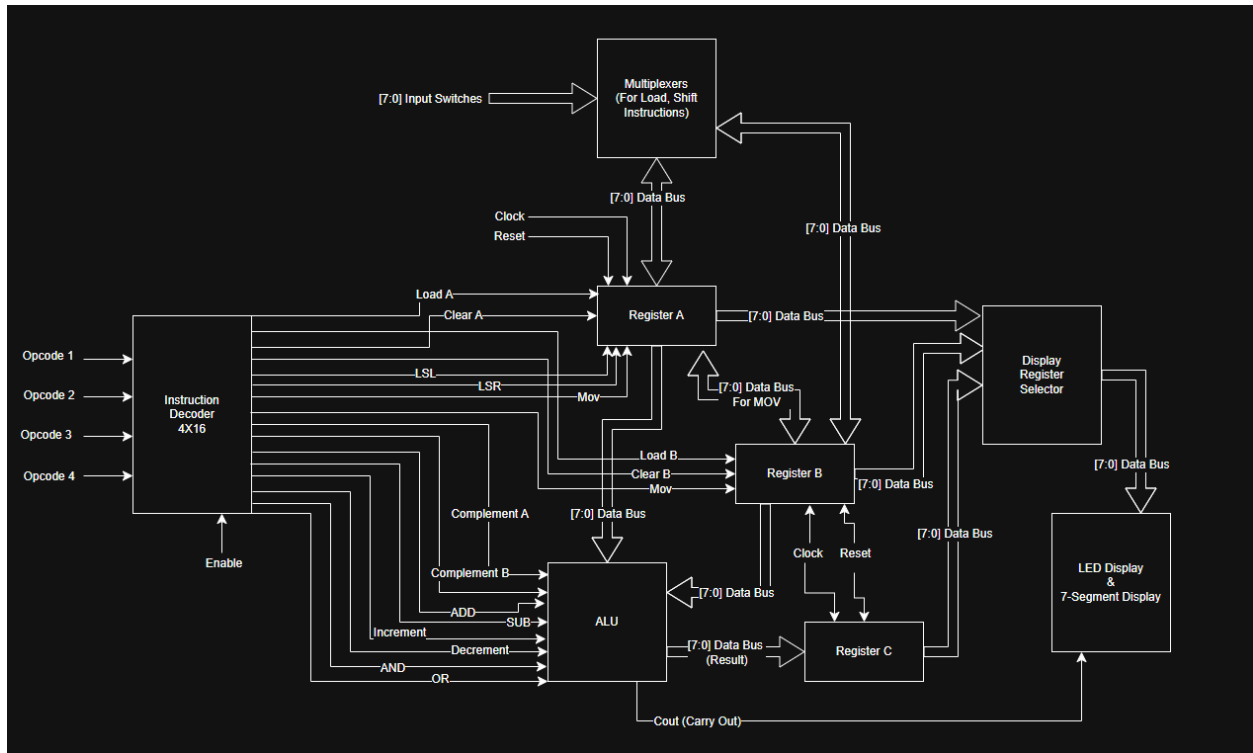


Figure 1: Block Diagram of Microprocessor

2.2 Instruction Set

For our instruction decoder, we will implement a 4X16 decoder to handle our 4-Bit opcodes and give one-hot output. This output will be a controlled signal that is generated by our instruction decoder and will tell our microprocessor which operation to perform. Below is the operations table for our instruction decoder.

Opcode	Label
0000	Load A
0001	Load B
0010	Clear A
0011	Clear B
0100	LSL A
0101	LSR A
0110	MOV $A \rightarrow B$
0111	MOV $B \rightarrow A$
1000	Complement A
1001	Complement B
1010	ADD
1011	SUB
1100	Increment
1101	Decrement
1110	AND
1111	OR

Table 1: Operation Table for Instruction Decoder

With our operations table, we can understand how our 4X16 instruction decoder is intended to work, and we can start designing it.

```

23 module Decoder_4x16(
24     input en,
25     input [3:0] a,
26     output reg [15:0] y
27 );
28
29 always @(*) begin
30     y = 16'b0000000000000000;
31     if (en) begin
32         case (a)
33             4'b0000: y = 16'b0000000000000001; // Load A
34             4'b0001: y = 16'b0000000000000010; // Load B
35             4'b0010: y = 16'b0000000000000100; // CLR A
36             4'b0011: y = 16'b0000000000000100; // CLR B
37             4'b0100: y = 16'b0000000000001000; // Logic Shift Left
38             4'b0101: y = 16'b0000000000001000; // Logic Shift Right
39             4'b0110: y = 16'b0000000001000000; // MOV A -> B
40             4'b0111: y = 16'b0000000001000000; // MOV B -> A
41             4'b1000: y = 16'b0000000010000000; // Complement A
42             4'b1001: y = 16'b0000000010000000; // Complement B
43             4'b1010: y = 16'b0000001000000000; // ADD
44             4'b1011: y = 16'b0000010000000000; // SUB
45             4'b1100: y = 16'b0001000000000000; // Increment A
46             4'b1101: y = 16'b0010000000000000; // Decrement A
47             4'b1110: y = 16'b0100000000000000; // AND
48             4'b1111: y = 16'b1000000000000000; // OR
49             default: y = 16'b0000000000000000;
50         endcase
51     end
52 end
53 endmodule

```

Figure 2: 4X16 Instruction Decoder Code

Once we've finished designing our 4X16 instruction decoder, we can open the elaborated design to see the gate-level schematic of our design.

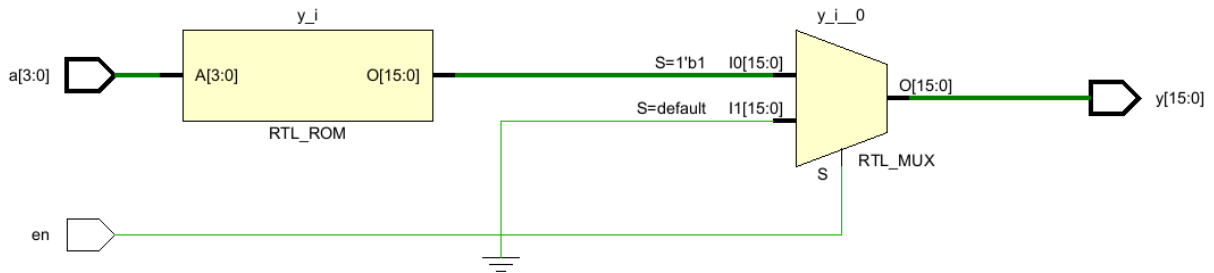


Figure 3: 4X16 Instruction Decoder Schematic

2.3 Debounce Logic

For our microprocessor, we will include a debounce logic module. The reason for the debounce logic is to ensure that when we press a pushbutton for the enable signal, it only pulses once. Without the debounce logic, the FPGA might unintentionally generate the enable signal multiple times when a pushbutton is pressed, which is due to the FPGA's high clock speed.

```
23 module Debounce_Logic(  
24     input clk,  
25     input button,  
26     output reg pulse  
27 );  
28  
29     // Synchronizer  
30     reg [1:0] sync;  
31     always @(posedge clk) begin  
32         sync <= {sync[1:0], button};  
33     end  
34  
35     // Debounce counter (~10ms for 100MHz clock)  
36     reg [19:0] counter = 0;  
37     reg stable = 0;  
38  
39     always @(posedge clk) begin  
40         if (sync[1] == stable)  
41             counter <= 0;  
42         else begin  
43             counter <= counter + 1;  
44             if (counter == 1_000_000) begin  
45                 stable <= sync[1];  
46                 counter <= 0;  
47             end  
48         end  
49     end  
50  
51     // Edge detect  
52     reg stable_d = 0;  
53     always @(posedge clk) begin  
54         stable_d <= stable;  
55         pulse <= stable & ~stable_d;  
56     end  
57 endmodule
```

Figure 4: Debounce Logic Code

When the debounce logic module is finished, we can open the elaborated design to see the gate-level schematic of our debounce logic module.

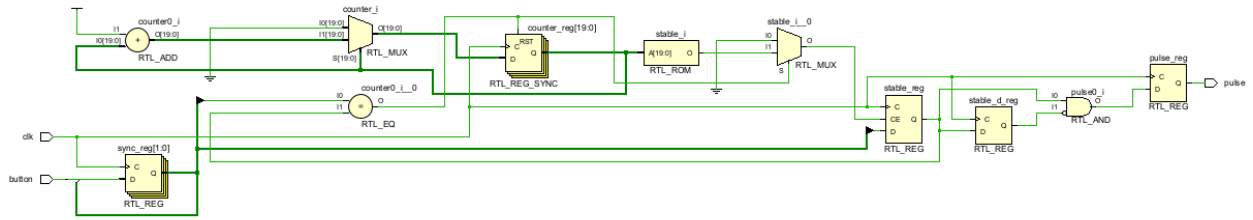


Figure 5: Debounce Module Schematic

2.4 8-Bit Microprocessor

Now that we have both the instruction decoder and debounce logic, we can start designing our microprocessor. We will do a case-based design for our 8-Bit microprocessor, where the control signal from our 4X16 instruction decoder determines which operations to execute. Since our design is case-based, we can implement a simple ALU to handle ADD, SUB, AND, OR, Increment, and Decrement operations. To display the values stored in our 8-Bit registers, we will utilize LEDs and a 7-segment display on our FPGA.

```
23 module Microprocessor_8Bit(  
24     input clk,  
25     input rst,  
26     input en,  
27     input [3:0] opcode,  
28     input [7:0] sw,  
29     input [1:0] RegSelect,  
30     output [7:0] LED,  
31     output [6:0] Segments,  
32     output [3:0] an,  
33     output reg Cout  
34 );  
35  
36 // Registers to Display Data  
37 reg [7:0] DisplayReg;  
38 reg [1:0] digit_select = 0;  
39 reg [6:0] segment_reg;  
40 reg [3:0] an_reg;  
41 reg [15:0] refresh_counter = 0;  
42  
43 // Registers to Store Data  
44 reg [7:0] RegA;  
45 reg [7:0] RegB;  
46 reg [7:0] RegC;  
47  
48 // Wires for 7-Segment Display  
49 wire [3:0] lower = DisplayReg[3:0];  
50 wire [3:0] upper = DisplayReg[7:4];  
51  
52 //Wire for Enable  
53 wire en_pulse;  
54  
55 //Wire for DCD  
56 wire [15:0] dec_out;  
57  
58 // Debounce Module  
59 Debounce_Logic u_db (  
60     .clk(clk),  
61     .button(en),  
62     .pulse(en_pulse)  
63 );
```

Figure 6: 8-Bit Microprocessor Code Part 1

```

66 | // 7-Segment Digit Select
67 | assign Segments = segment_reg;
68 | assign an = an_reg;
69 | always @(posedge clk) begin
70 |     refresh_counter <= refresh_counter + 1;
71 |     digit_select <= refresh_counter[15:14];
72 | end
73 |
74 | always @(*) begin
75 |     case(digit_select)
76 |     2'b00: begin
77 |         an_reg = 4'b1110;
78 |         segment_reg = hex_to_7segment(lower);
79 |     end
80 |     2'b01: begin
81 |         an_reg = 4'b1101;
82 |         segment_reg = hex_to_7segment(upper);
83 |     end
84 |     default: begin
85 |         an_reg = 4'b1111;
86 |         segment_reg = 7'b1111111;
87 |     end
88 | endcase
89 | end
90 |
91 | // Decoder
92 | Decoder_4x16 u_dec(
93 |     .en(en_pulse),
94 |     .a(opcode),
95 |     .y(dec_out)
96 | );
97 |

```

Figure 7: 8-Bit Microprocessor Code Part 2

```

99 : // Display Register Select
100 ⊞ always @(*) begin
101 ⊞     case(RegSelect)
102 :         2'b01: DisplayReg = RegA;
103 :         2'b10: DisplayReg = RegB;
104 :         2'b11: DisplayReg = RegC;
105 :         default: DisplayReg = 8'b00000000;
106 ⊞     endcase
107 ⊞ end
108 : assign LED[7:0] = DisplayReg;
109 :
110 :
111 : // 7 Segment Display
112 ⊞ function [6:0] hex_to_7segment;
113 :     input [3:0] hex;
114 ⊞     case(hex)
115 :         4'b0000: hex_to_7segment = 7'b1000000;
116 :         4'b0001: hex_to_7segment = 7'b1111001;
117 :         4'b0010: hex_to_7segment = 7'b0100100;
118 :         4'b0011: hex_to_7segment = 7'b0110000;
119 :         4'b0100: hex_to_7segment = 7'b0011001;
120 :         4'b0101: hex_to_7segment = 7'b0010010;
121 :         4'b0110: hex_to_7segment = 7'b0000010;
122 :         4'b0111: hex_to_7segment = 7'b1111000;
123 :         4'b1000: hex_to_7segment = 7'b0000000;
124 :         4'b1001: hex_to_7segment = 7'b0010000;
125 :         4'b1010: hex_to_7segment = 7'b0001000;
126 :         4'b1011: hex_to_7segment = 7'b0000011;
127 :         4'b1100: hex_to_7segment = 7'b1000110;
128 :         4'b1101: hex_to_7segment = 7'b0100001;
129 :         4'b1110: hex_to_7segment = 7'b0000110;
130 :         4'b1111: hex_to_7segment = 7'b0001110;
131 ⊞     endcase
132 ⊞ endfunction
133 :

```

Figure 8: 8-Bit Microprocessor Code Part 3

```

135 : // Main Logic
136 : always @(posedge clk or posedge rst) begin
137 :     if (rst) begin
138 :         RegA <= 8'b00000000;
139 :         RegB <= 8'b00000000;
140 :         RegC <= 8'b00000000;
141 :         Cout <= 1'b0;
142 :     end
143 :
144 :     else begin
145 :         if (dec_out[0]) begin
146 :             RegA <= sw; // Load A
147 :         end
148 :         else if (dec_out[1]) begin
149 :             RegB <= sw; // Load B
150 :         end
151 :         else if (dec_out[2]) begin
152 :             RegA <= 8'b00000000; // CLR A
153 :         end
154 :         else if (dec_out[3]) begin
155 :             RegB <= 8'b00000000; // CLR B
156 :         end
157 :         else if (dec_out[4]) begin
158 :             RegA <= {RegA[6:0], 1'b0}; // LSL A
159 :         end
160 :         else if (dec_out[5]) begin
161 :             RegA <= {1'b0, RegA[7:1]}; // LSR A
162 :         end
163 :         else if (dec_out[6]) begin
164 :             RegB <= RegA; // MOV A -> B
165 :         end
166 :         else if (dec_out[7]) begin
167 :             RegA <= RegB; // MOV B -> A
168 :         end
169 :         else if (dec_out[8]) begin
170 :             RegA <= ~RegA; // Complement A
171 :         end
172 :         else if (dec_out[9]) begin
173 :             RegB <= ~RegB; // Complement B
174 :         end

```

Figure 9: 8-Bit Microprocessor Code Part 4

```

175 :         else if (dec_out[10]) begin
176 :             {Cout, RegC} <= RegA + RegB; // ADD
177 :         end
178 :         else if (dec_out[11]) begin
179 :             {Cout, RegC} <= RegA - RegB; // SUB
180 :         end
181 :         else if (dec_out[12]) begin
182 :             RegA <= RegA + 1; // Increment A
183 :         end
184 :         else if (dec_out[13]) begin
185 :             RegA <= RegA - 1; // Decrement A
186 :         end
187 :         else if (dec_out[14]) begin
188 :             RegC <= RegA & RegB; // AND
189 :         end
190 :         else if (dec_out[15]) begin
191 :             RegC <= RegA | RegB; // OR
192 :         end
193 :     end
194 : end
195 :
196 : endmodule

```

Figure 10: 8-Bit Microprocessor Code Part 5

Once we've finished designing our 8-Bit microprocessor, we can open the elaborated design to see the gate-level schematic of our microprocessor.

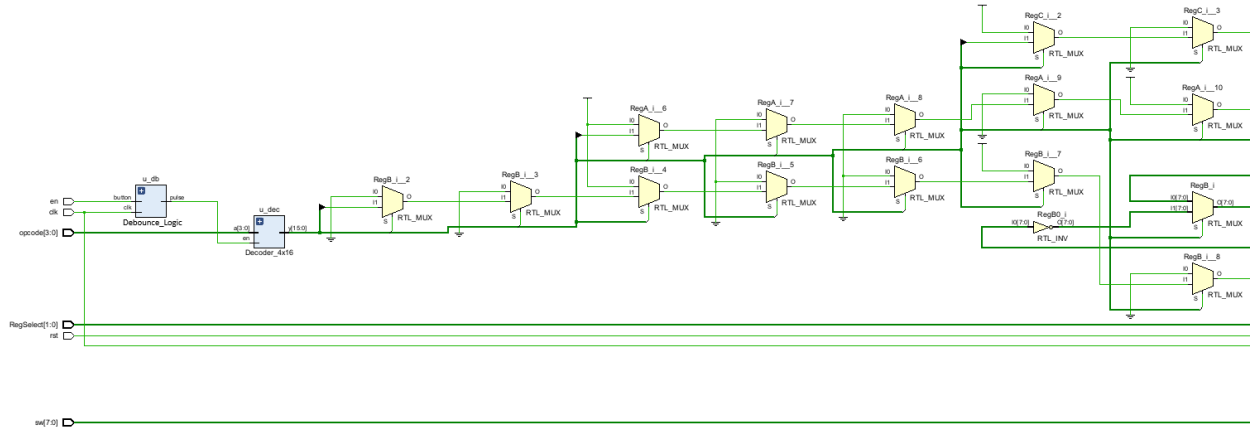


Figure 11: 8-Bit Microprocessor Schematic Part 1

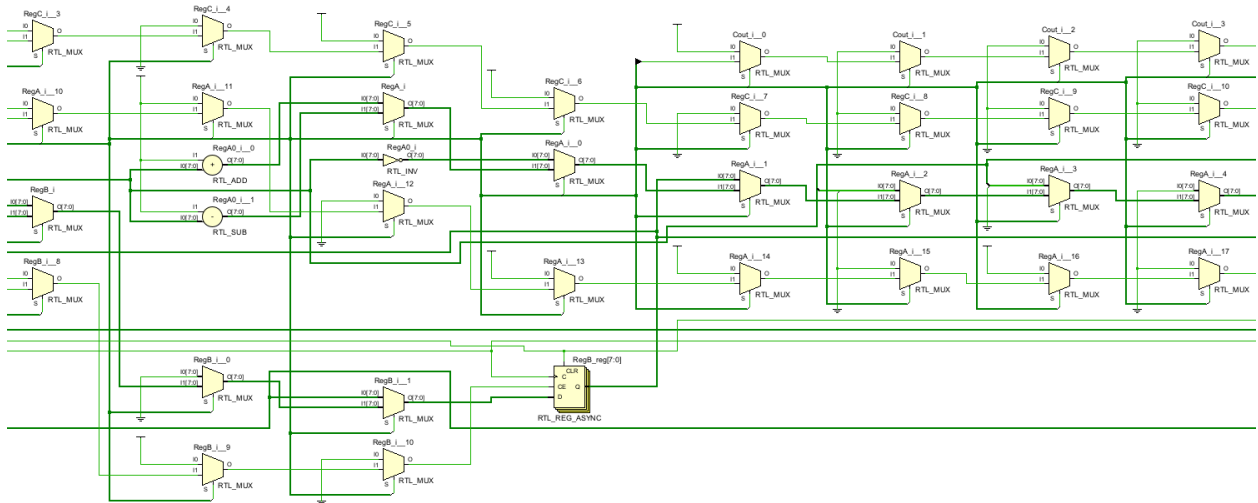


Figure 12: 8-Bit Microprocessor Schematic Part 2

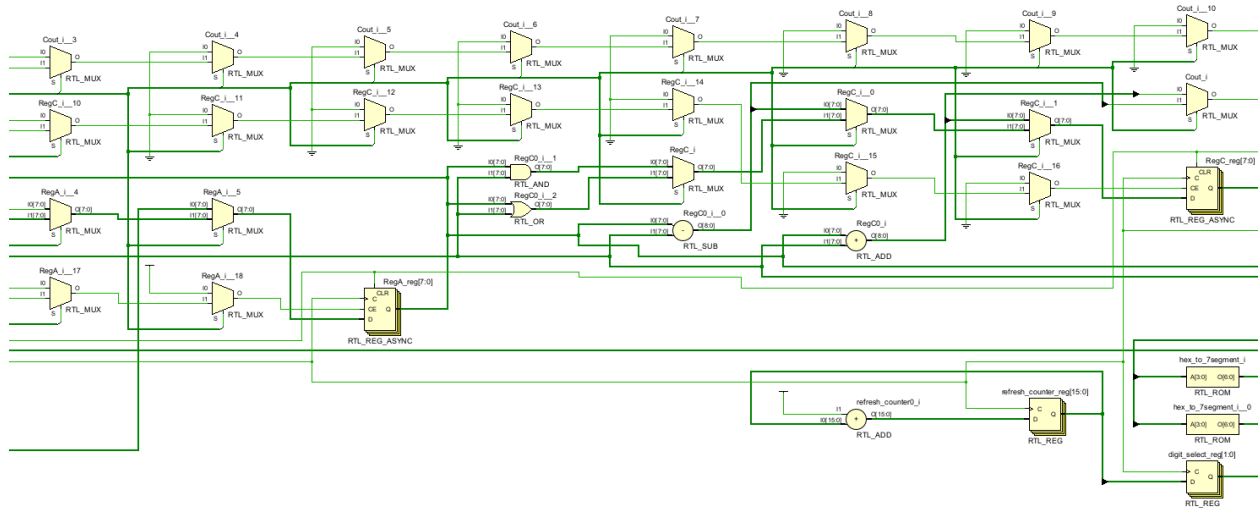


Figure 13: 8-Bit Microprocessor Schematic Part 3

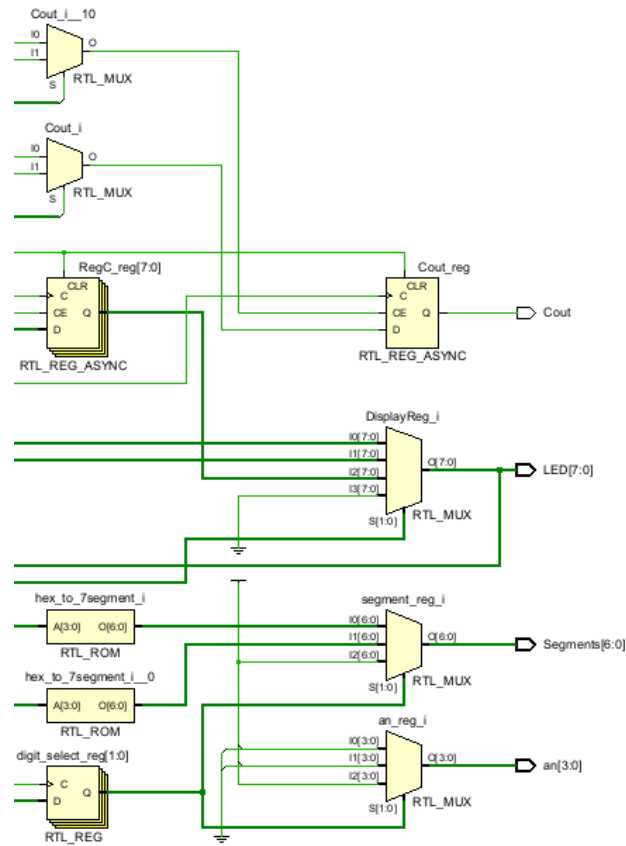


Figure 14: 8-Bit Microprocessor Schematic Part 4

3 Design Testing

3.1 Behavioural Simulation

For our testbench, we will implement a case-based testing sequence. In this way, we can simulate each operation based on the generated opcodes. To check the correctness of each operation, we will compare the generated output with the expected output. If they don't match, then the operation has failed. We will also use the command `$display(...)` to display a string onto the TCL Console, which will tell us if the operation was successful or not. By using the TCL Console in addition to the waveform, we can verify each operation in a much faster manner.

```
23 | module Microprocessor_8Bit_tb;
24 | reg clk;
25 | reg rst;
26 | reg en;
27 | reg [3:0] opcode;
28 | reg [7:0] sw;
29 | reg [1:0] RegSelect;
30 |
31 | wire [7:0] LED;
32 | wire Cout;
33 |
34 |
35 | Microprocessor_8Bit dut (
36 |     .clk(clk),
37 |     .rst(rst),
38 |     .en(en),
39 |     .opcode(opcode),
40 |     .sw(sw),
41 |     .Cout(Cout),
42 |     .RegSelect(RegSelect),
43 |     .LED(LED)
44 | );
45 |
46 | // Clock Generation
47 | initial begin
48 |     clk = 1'b0;
49 |     forever #5 clk = ~clk;
50 | end
```

Figure 15: Testbench Part 1


```

52 // Execute Instructions
53 task exec_instruction;
54     input [3:0] op;
55     input [7:0] data;
56     begin
57         @(negedge clk);
58         opcode = op;
59         sw = data;
60         en = 1'b1;
61
62         @(posedge clk);
63         en = 1'b0;
64
65         @(posedge clk);
66     end
67 endtask
68
69 // Test sequence
70 initial begin
71
72     rst = 1;
73     en = 0;
74     opcode = 0;
75     sw = 0;
76     RegSelect = 0;
77
78     repeat(2) @(posedge clk);
79     rst = 0;
80
81     // ----- LOAD A -----
82     RegSelect = 2'b01;
83     exec_instruction(4'b0000, 8'b00111100);
84     if (LED != 8'b00111100) $display("Error: LOAD A FAILED");
85     else $display("PASS: LOAD A");
86
87     // ----- LOAD B -----
88     RegSelect = 2'b10;
89     exec_instruction(4'b0001, 8'b11100011);
90     if (LED != 8'b11100011) $display("Error: LOAD B FAILED");
91     else $display("PASS: LOAD B");

```

Figure 16: Testbench Part 2

```

93 // ----- CLR A -----
94 RegSelect = 2'b01;
95 exec_instruction(4'b0010, 8'b00000000);
96 if (LED != 8'b00000000) $display("Error: CLR A FAILED");
97 else $display("PASS: CLR A");
98
99 // ----- CLR B -----
100 RegSelect = 2'b10;
101 exec_instruction(4'b0011, 8'b00000000);
102 if (LED != 8'b00000000) $display("Error: CLR B FAILED");
103 else $display("PASS: CLR B");
104
105 // ----- LSL A -----
106 RegSelect = 2'b01;
107 exec_instruction(4'b0000, 8'b00101100);
108 exec_instruction(4'b0100, 8'b00000000);
109 if (LED != 8'b01011000) $display("Error: LSL A FAILED");
110 else $display("PASS: LSL A");
111
112 // ----- LSR A -----
113 RegSelect = 2'b01;
114 exec_instruction(4'b0000, 8'b00101100);
115 exec_instruction(4'b0101, 8'b00000000);
116 if (LED != 8'b00010110) $display("Error: LSR A FAILED");
117 else $display("PASS: LSR A");
118
119 // ----- MOV A->B -----
120 RegSelect = 2'b01;
121 exec_instruction(4'b0000, 8'b11110000);
122 RegSelect = 2'b10;
123 exec_instruction(4'b0110, 8'b00000000);
124 if (LED != 8'b11110000) $display("Error: MOV A->B FAILED");
125 else $display("PASS: MOV A->B");
126
127 // ----- MOV B->A -----
128 RegSelect = 2'b10;
129 exec_instruction(4'b0001, 8'b00001111);
130 RegSelect = 2'b01;
131 exec_instruction(4'b0111, 8'b00000000);
132 if (LED != 8'b00001111) $display("Error: MOV B->A FAILED");
133 else $display("PASS: MOV B->A");

```

Figure 17: Testbench Part 3

```

135 // ----- Complement A -----
136 RegSelect = 2'b01;
137 exec_instruction(4'b0000, 8'b10100101);
138 exec_instruction(4'b1000, 8'b00000000);
139 if (LED != 8'b01011010) $display("Error: Complement A FAILED");
140 else $display("PASS: Complement A");
141
142 // ----- Complement B -----
143 RegSelect = 2'b10;
144 exec_instruction(4'b0001, 8'b01011010);
145 exec_instruction(4'b1001, 8'b00000000);
146 if (LED != 8'b10100101) $display("Error: Complement B FAILED");
147 else $display("PASS: Complement B");
148
149 // ----- ADD -----
150 RegSelect = 2'b01;
151 exec_instruction(4'b0000, 8'b10100010);
152 RegSelect = 2'b10;
153 exec_instruction(4'b0001, 8'b11001100);
154 exec_instruction(4'b1010, 8'b00000000);
155 RegSelect = 2'b11;
156 @(posedge clk);
157 if (LED != 8'b01101110 || Cout != 1)
158     $display("Error: ADD FAILED");
159 else $display("PASS: ADD");
160
161 // ----- SUB -----
162 RegSelect = 2'b01;
163 exec_instruction(4'b0000, 8'b10100010);
164 RegSelect = 2'b10;
165 exec_instruction(4'b0001, 8'b11001100);
166 exec_instruction(4'b1011, 8'b00000000);
167 RegSelect = 2'b11;
168 @(posedge clk);
169 if (LED != 8'b11010110 || Cout != 1)
170     $display("Error: SUB FAILED");
171 else $display("PASS: SUB");

```

Figure 18: Testbench Part 4

```

173 // ----- Increment A -----
174 RegSelect = 2'b01;
175 exec_instruction(4'b0000, 8'b01100110);
176 exec_instruction(4'b1100, 8'b00000000);
177 if (LED != 8'b01100111)
178     $display("Error: Increment A FAILED");
179 else $display("PASS: Increment A");
180
181 // ----- Decrement A -----
182 RegSelect = 2'b01;
183 exec_instruction(4'b0000, 8'b10011001);
184 exec_instruction(4'b1101, 8'b00000000);
185 if (LED != 8'b10011000)
186     $display("Error: Decrement A FAILED");
187 else $display("PASS: Decrement A");
188
189 // ----- AND -----
190 RegSelect = 2'b01;
191 exec_instruction(4'b0000, 8'b00100011);
192 RegSelect = 2'b10;
193 exec_instruction(4'b0001, 8'b01100010);
194 exec_instruction(4'b1110, 8'b00000000);
195 RegSelect = 2'b11;
196 @(posedge clk);
197 if (LED != 8'b00100010)
198     $display("Error: AND FAILED");
199 else $display("PASS: AND");
200
201 // ----- OR -----
202 RegSelect = 2'b01;
203 exec_instruction(4'b0000, 8'b00100011);
204 RegSelect = 2'b10;
205 exec_instruction(4'b0001, 8'b01100010);
206 exec_instruction(4'b1111, 8'b00000000);
207 RegSelect = 2'b11;
208 @(posedge clk);
209 if (LED != 8'b01100011)
210     $display("Error: OR FAILED");
211 else $display("PASS: OR");
212
213 $display("Test Completed");
214 $stop;
215 end

```

Figure 19: Testbench Part 5

Once our testbench is completed, we can run a behavioural simulation to obtain our waveform and results from the TCL Console. Once ran, we can observe the waveform and the TCL Console to see if any operations fail or pass.

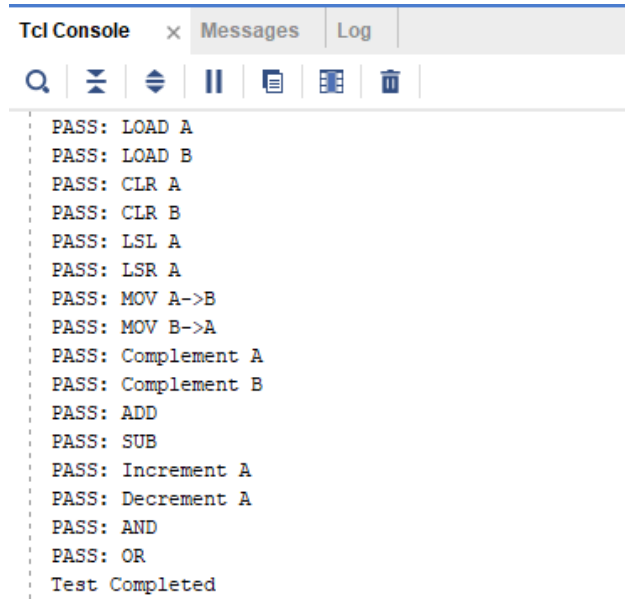


Figure 20: TCL Console Text Display

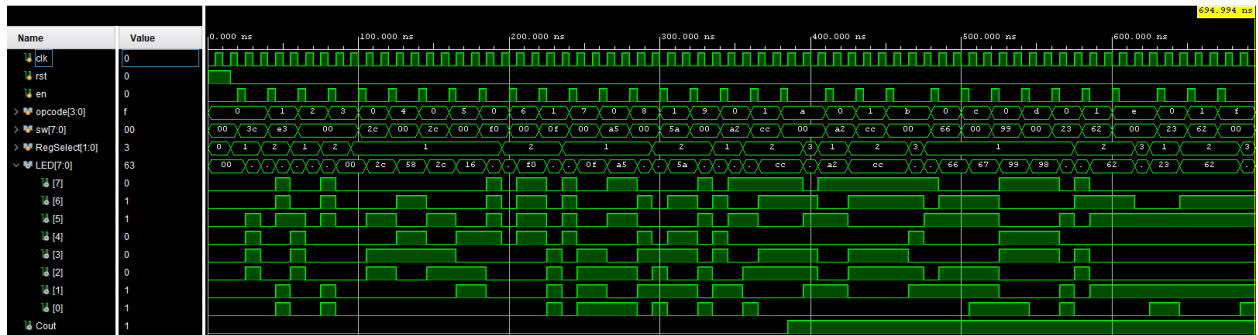


Figure 21: 8-Bit Microprocessor Waveform

From the waveform and TCL Console, we can see that all operations passed and are working as expected.

3.1.1 Error Encounter

During simulating our testbench, we've encountered an error where "LED" was constantly outputting "00". The issue was found to be the enable signal, caused by the DebounceLogic module. Within the DebounceLogic module, the enable pulse that is generated is set to happen for every 1,000,000 clock cycles. This was implemented for hardware testing, but it is an issue when simulating digitally because the testbench only pulses the enable for 1-2 clock cycles. Due to this issue, enable was never asserted, and the instruction decoder was always off. We've resolved this issue by passing a constant enable signal through the instruction decoder only for simulation purposes.

3.2 FPGA Implementation

Now that we have finished simulating our design, we can implement our design on an FPGA board. We will now write our constraints onto an XDC file. This XDC file will map our physical ports and pins so that we can physically test our design.

```
1 set_property CFGBVS VCC0 [current_design]
2
3 set_property CONFIG_VOLTAGE 3.3 [current_design]
4
5 # Clock
6 set_property -dict { PACKAGE_PIN W5 IOSTANDARD LVCMOS33 } [get_ports { clk }];
7 create_clock -period 10.000 [get_ports { clk }];
8
9 # EN Switch
10 set_property -dict { PACKAGE_PIN T18 IOSTANDARD LVCMOS33 } [get_ports { en }];
11
12 # Reset Button
13 set_property -dict { PACKAGE_PIN W19 IOSTANDARD LVCMOS33 } [get_ports { rst }];
14
15 # Input Switches
16 set_property -dict { PACKAGE_PIN V17 IOSTANDARD LVCMOS33 } [get_ports { sw[0] }];
17 set_property -dict { PACKAGE_PIN V16 IOSTANDARD LVCMOS33 } [get_ports { sw[1] }];
18 set_property -dict { PACKAGE_PIN W16 IOSTANDARD LVCMOS33 } [get_ports { sw[2] }];
19 set_property -dict { PACKAGE_PIN W17 IOSTANDARD LVCMOS33 } [get_ports { sw[3] }];
20 set_property -dict { PACKAGE_PIN W15 IOSTANDARD LVCMOS33 } [get_ports { sw[4] }];
21 set_property -dict { PACKAGE_PIN V15 IOSTANDARD LVCMOS33 } [get_ports { sw[5] }];
22 set_property -dict { PACKAGE_PIN W14 IOSTANDARD LVCMOS33 } [get_ports { sw[6] }];
23 set_property -dict { PACKAGE_PIN W13 IOSTANDARD LVCMOS33 } [get_ports { sw[7] }];
24
25 # Opcode Switches
26 set_property -dict { PACKAGE_PIN T3 IOSTANDARD LVCMOS33 } [get_ports { opcode[0] }];
27 set_property -dict { PACKAGE_PIN T2 IOSTANDARD LVCMOS33 } [get_ports { opcode[1] }];
28 set_property -dict { PACKAGE_PIN R3 IOSTANDARD LVCMOS33 } [get_ports { opcode[2] }];
29 set_property -dict { PACKAGE_PIN W2 IOSTANDARD LVCMOS33 } [get_ports { opcode[3] }];
30
31 # Register Select Switches
32 set_property -dict { PACKAGE_PIN T1 IOSTANDARD LVCMOS33 } [get_ports { RegSelect[0] }];
33 set_property -dict { PACKAGE_PIN R2 IOSTANDARD LVCMOS33 } [get_ports { RegSelect[1] }];
```

Figure 22: XDC Constraints Part 1

```

35 # Display LED
36 set_property -dict { PACKAGE_PIN U16 IOSTANDARD LVCMOS33 } [get_ports { LED[0] }];
37 set_property -dict { PACKAGE_PIN E19 IOSTANDARD LVCMOS33 } [get_ports { LED[1] }];
38 set_property -dict { PACKAGE_PIN U19 IOSTANDARD LVCMOS33 } [get_ports { LED[2] }];
39 set_property -dict { PACKAGE_PIN V19 IOSTANDARD LVCMOS33 } [get_ports { LED[3] }];
40 set_property -dict { PACKAGE_PIN W18 IOSTANDARD LVCMOS33 } [get_ports { LED[4] }];
41 set_property -dict { PACKAGE_PIN U15 IOSTANDARD LVCMOS33 } [get_ports { LED[5] }];
42 set_property -dict { PACKAGE_PIN U14 IOSTANDARD LVCMOS33 } [get_ports { LED[6] }];
43 set_property -dict { PACKAGE_PIN V14 IOSTANDARD LVCMOS33 } [get_ports { LED[7] }];
44 set_property -dict { PACKAGE_PIN L1 IOSTANDARD LVCMOS33 } [get_ports { Cout }];
45
46 # Display 7-Segment
47 set_property -dict { PACKAGE_PIN W7 IOSTANDARD LVCMOS33 } [get_ports { Segments[0] }];
48 set_property -dict { PACKAGE_PIN W6 IOSTANDARD LVCMOS33 } [get_ports { Segments[1] }];
49 set_property -dict { PACKAGE_PIN U8 IOSTANDARD LVCMOS33 } [get_ports { Segments[2] }];
50 set_property -dict { PACKAGE_PIN V8 IOSTANDARD LVCMOS33 } [get_ports { Segments[3] }];
51 set_property -dict { PACKAGE_PIN U5 IOSTANDARD LVCMOS33 } [get_ports { Segments[4] }];
52 set_property -dict { PACKAGE_PIN V5 IOSTANDARD LVCMOS33 } [get_ports { Segments[5] }];
53 set_property -dict { PACKAGE_PIN U7 IOSTANDARD LVCMOS33 } [get_ports { Segments[6] }];
54
55 set_property -dict { PACKAGE_PIN U2 IOSTANDARD LVCMOS33 } [get_ports { an[0] }];
56 set_property -dict { PACKAGE_PIN U4 IOSTANDARD LVCMOS33 } [get_ports { an[1] }];
57 set_property -dict { PACKAGE_PIN V4 IOSTANDARD LVCMOS33 } [get_ports { an[2] }];
58 set_property -dict { PACKAGE_PIN W4 IOSTANDARD LVCMOS33 } [get_ports { an[3] }];

```

Figure 23: XDC Constraints Part 2

Once we've coded our XDC file, we can now run synthesis and implementation. We will also run an implemented design to see how our 8-Bit microprocessor is mapped onto the FPGA.

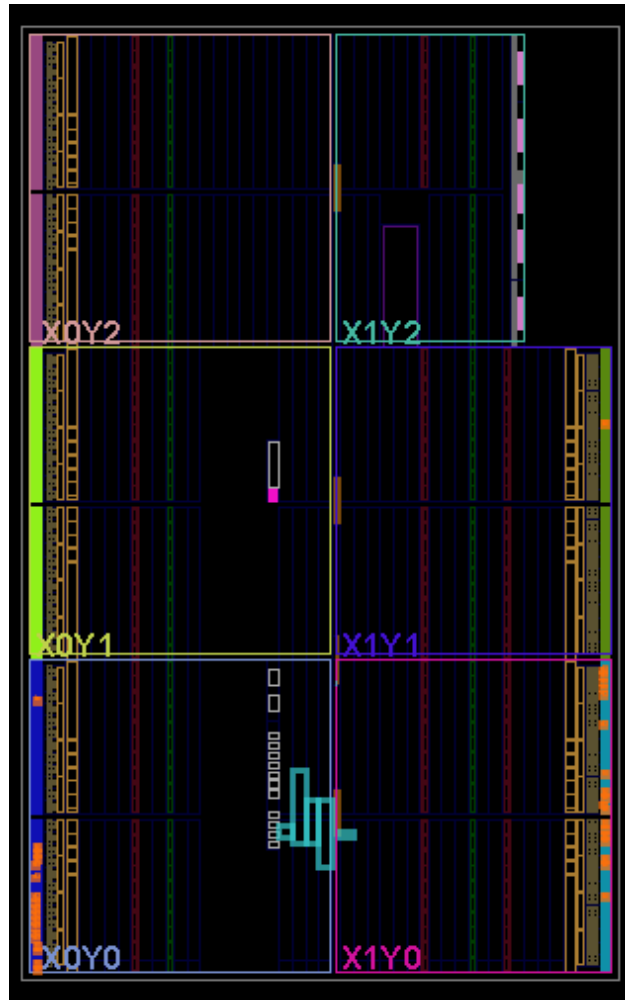


Figure 24: FPGA Implemented Design

Now that we've finished running synthesis and implementation, we can generate a bitstream and program our FPGA to physically test our design. For our FPGA, we will be using the BASYS 3 FPGA. Below shows where our inputs are, and how our outputs are going to be displayed.

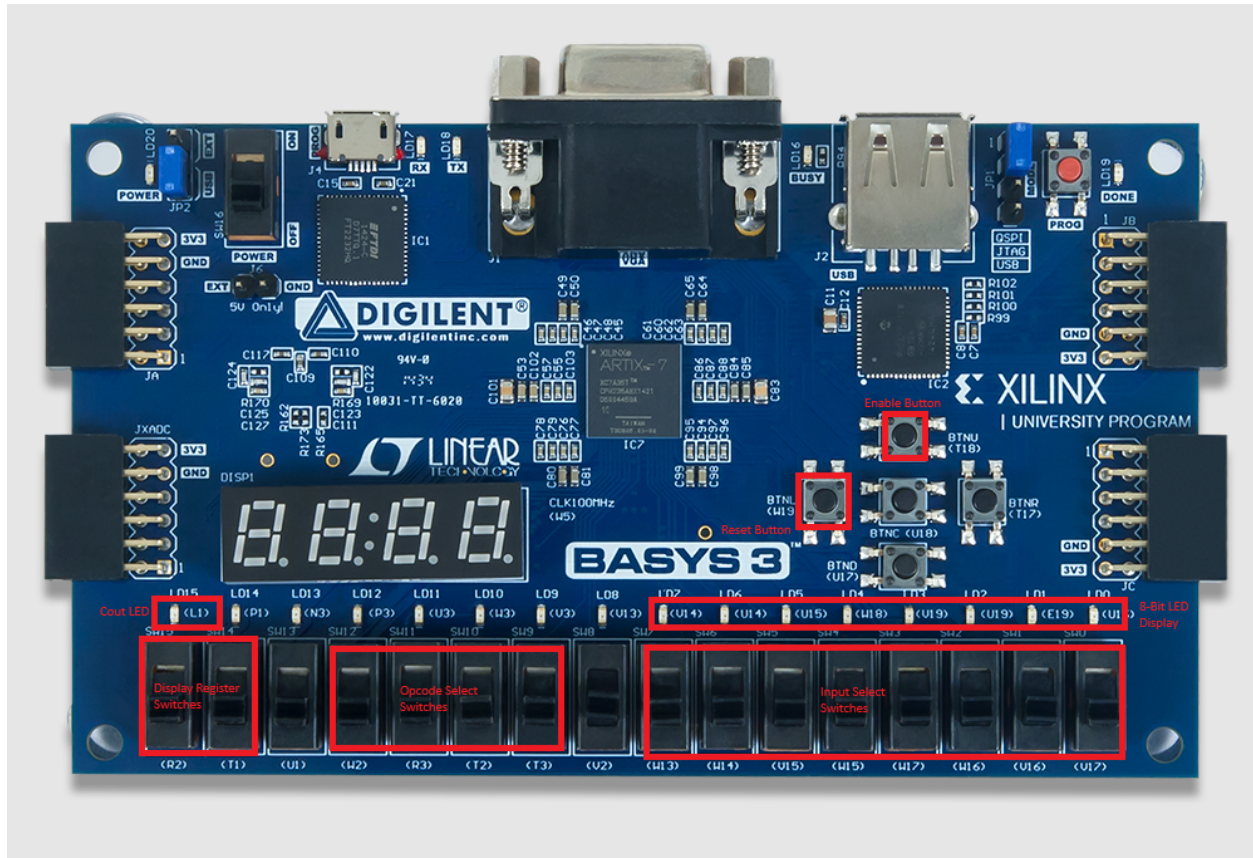


Figure 25: BASYS 3 FPGA Mapping

4 Conclusion

In this project, we successfully designed and implemented a functional 8-bit microprocessor with an instruction decoder and an arithmetic logic unit. The processor can execute basic arithmetic, logical, and data transfer operations, demonstrating the core principles of microprocessor operation. Through this implementation, we gained hands-on experience in instruction decoding, data path design, and control logic, reinforcing foundational concepts in computer architecture. Despite being an older technology, 8-bit microprocessors remain highly practical in real-world applications, including embedded systems, automotive controls, industrial automation, and low-power IoT devices. This project not only illustrates the enduring relevance of 8-bit processors but also provides a framework for further exploration, such as adding memory management, expanding the instruction set, or integrating peripheral interfaces. Overall, the design showcases how fundamental microprocessor principles can be applied to real-world systems that prioritize simplicity, efficiency, and cost-effectiveness.